

React Fundamentals & Core Concepts

Introduction

What is React?

- React is a **JavaScript** library and open source project created by **Facebook**
- React is used to build interactive **user interfaces (UI)** components.
- One of the most important aspects of React is the fact that you can create components, which are like custom, reusable HTML elements, to quickly and efficiently build user interfaces. React also streamlines how data is stored and handled, using state and props.
- Popular sites, such as **Instagram, Facebook, Uber, Netflix, Airbnb** etc uses ReactJS.
- Because of its ability to create fast, efficient, and scalable web applications, React has gained stability and popularity. Thousands of web applications use it today, from well-established companies to new start-ups.

History

- React was first designed by **Jorden Walke**, a software engineer at Facebook.
- It was first deployed for Facebook News feed around 2011.
- In 2013, React was open sourced at JS conference.

About React

- **Component based approach:** A component is one of the core building blocks of React. In other words, we can say that every application you will develop in react will be made up of pieces called components. Components make the task of building UIs much easier.
- **Declarative approach:** Declarative programming is a programming paradigm that expresses the logic of a computation without describing its control flow.

Key Features

- Component reusability
- Virtual DOM for efficient rendering
- Unidirectional data flow using props
- Strong ecosystem and developer tools

ReactJS Virtual DOM

It is an in-memory representation of the actual DOM (Document Object Model). React uses this lightweight JavaScript object to track changes in the application state and efficiently update the actual DOM only where necessary.

What is the Virtual DOM?

The Virtual DOM (VDOM) is a lightweight, in-memory representation of the real DOM (Document Object Model). It helps React manage UI updates more efficiently by keeping a virtual version of the UI in memory. When changes occur, React updates only the necessary parts of the real DOM, instead of re-rendering everything.

How Does the Virtual DOM Work?

Rendering the Virtual DOM: React creates a virtual representation of the UI as a tree of JavaScript objects.

Updating State: It generates a new Virtual DOM tree to reflect the updated state when the application state changes.

Diffing Algorithm: React compares the new Virtual DOM tree with the previous one using its efficient diffing algorithm to identify the minimal set of changes required.

Updating the Real DOM: React applies only the necessary changes to the real DOM, optimizing rendering performance.

React Single Page Application (SPA)

In a React Single Page Application (SPA), smooth navigation relies on the **Virtual DOM** and the `<Link>` component from `react-router-dom`.

How Does it works?

Initial Load: React builds the entire UI (navbar, Home content, footer) as a **Virtual DOM** blueprint. The browser then draws the **Real DOM** from this blueprint.

- **Navigation (e.g., clicking "About"):**
 - You **must use** `<Link to="/about">`, not `<a>`. `<Link>` prevents a full page reload.
 - React Router updates the URL and triggers a re-render in React.
 - React creates a **NEW Virtual DOM** for the "About" page.
 - React's "diffing" algorithm compares the new "About" Virtual DOM with the old "Home" one. It sees the navbar and footer are identical, but the content has changed.
 - React then tells the browser to **only update the changed part of the Real DOM** (just the content area).

Setup of React JS

Step 1: Install Node.js installer for windows.

Step 2: Open command prompt or terminal in VS code to check whether it is completely installed or not.

```
The command -> node -v  
npm -v
```

If the installation went well it will give you the version you have installed

Step 3: Now Create a new folder where you want to make your react app using the below command:

```
mkdir react
```

***Note:** The **react** in the above command is the name of the folder and can be anything.*

Move inside the same folder using the below command:

```
cd react
```

Step 4: Now, go inside **folder(react)**.

Install Create React App using Vite

1. Create Vite Project

```
npm create vite@latest
```

2. Enter Project Name

Example: **myapp**

Creates a folder named myapp.

```
> npx  
> create-vite  
  
◆ Project name:  
  vite-project
```

3. Select Framework

Choose: **React**

Used to create a React application.

```
◆ Project name:  
  myapp  
  
◆ Select a framework:  
  ● Vanilla  
  ○ Vue  
  ○ React  
  ○ Preact  
  ○ Lit  
  ○ Svelte  
  ○ Solid  
  ○ Ember  
  ○ Qwik  
  ○ Angular  
  ○ Marko  
  ○ Others
```

4. Select Variant

Choose: **JavaScript**

- Easy for beginners
- Most tutorials use this

```
◇ Project name:
  myapp
◇ Select a framework:
  React
◆ Select a variant:
  ○ TypeScript
  ○ TypeScript + React Compiler
  ● JavaScript
  ○ JavaScript + React Compiler
  ○ RSC
  ○ React Router v7 / https://reactrouter.com
  ○ TanStack Router / https://tanstack.com/router
  ○ RedwoodSDK / https://rwsdk.com
  ○ Vike / https://vike.dev
```

JavaScript + React Compiler is an advanced optimized version.

5. Vite asks to start Install with npm and start now?

```
◇ Project name:
  myapp
◇ Select a framework:
  React
◇ Select a variant:
  JavaScript
◆ Install with npm and start now?
  ● Yes / ○ No
```

If you choose: **Yes**

Vite automatically runs:
npm install
npm run dev

So the development server starts automatically.

6. Open in Browser

<http://localhost:5173/>

Default Vite development URL.

If port 5173 is already in use, Vite automatically uses another port like:

5174

5175

Example:

Port 5173 is in use, trying another one...

Shortcut Command

```
npm create vite@latest myapp -- --template react
```

Creates a React project directly without asking questions.

Part	Meaning
Npm	Node package manager
Create	Create a new project
vite@latest	Use latest Vite
Myapp	Project folder name
--	Pass options to Vite
--template react	Create a React project

Useful Vite Commands**Install and Start Project**

```
cd project-name  
npm install  
npm run dev
```

- npm install → installs packages
- npm run dev → starts Vite server

Run on Different Port

```
npm run dev -- --port 3000
```

Stop Server

```
Ctrl + C
```

Build Project

```
npm run build  
Creates production files.
```

Preview Build

```
npm run preview
```

Preview production version locally.

Folder Structure

Step-by-Step Rendering Process

Step 1:

Browser first opens index.html. This file contains the root container where the React app will be displayed.

Step 2:

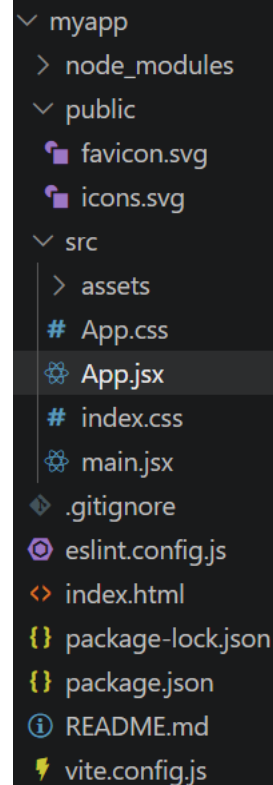
main.jsx loads React and connects the application with the browser DOM.

Step 3:

App.jsx renders the actual user interface such as headings, buttons, images and components.

Rendering Flow:

index.html → main.jsx → App.jsx → Browser Screen



```

myapp
├── node_modules
├── public
│   ├── favicon.svg
│   └── icons.svg
├── src
│   ├── assets
│   ├── App.css
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── .gitignore
├── eslint.config.js
├── index.html
├── package-lock.json
├── package.json
├── README.md
└── vite.config.js
  
```

Note: For Detailed Folder structure refer Annexure -I at the end

index.html

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vite + React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
  
```

HTML Setup

Basic HTML structure for React app.

- <!doctype html> → Defines HTML5 document.
- <html lang="en"> → Sets language as English.

Head Section

Contains webpage information.

- meta charset="UTF-8" and viewport → Supports all characters and makes page mobile responsive.
 - title → Sets browser tab name.
-

Root Element

```
<div id="root"></div>
```

- Main container for React app.
 - React displays all components inside this div.
-

Script Loading

```
<script type="module" src="/src/main.jsx"></script>
```

- Loads main.jsx
 - Starts React application.
-

Working

1. Browser loads index.html
 2. Finds root div
 3. Loads main.jsx
 4. React displays app inside root
-

main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
)
```

Import Statements

React features and files are imported at the top.

- StrictMode helps detect warnings.
 - createRoot renders React into browser.
 - index.css applies global styling.
 - App.jsx imports main component.
-

Root Element Selection

document.getElementById('root') finds the root div from index.html.

Rendering React

.render() displays the App component inside browser.

Working

1. main.jsx runs first
 2. Finds root div
 3. Imports App component
 4. Renders App inside browser
-

App.jsx

```
function App() {  
  return (  
    <div>  
      <h1>Welcome to React with Vite</h1>  
      <p>This is App component</p>  
    </div>  
  )  
}  
export default App
```

Function Component

React component is created using function.

- function App() → Creates App component.
-

Return Statement

- return() → Returns JSX (HTML-like code).
-

JSX Content

UI elements written inside return.

- <div> → Parent container.
 - <h1> → Displays heading.
 - <p> → Displays paragraph.
-

Export Statement

- export default App → Makes App available to other files.
-

Working

1. App component creates webpage UI
 2. main.jsx imports App
 3. React displays App in browser
-

React Components

- A react component is a JavaScript function that you can sprinkle with markup
- React lets you create components, reusable UI elements for your app.
- In a React app, every piece of UI is a component.
- React components are regular JavaScript functions except:
 - a. Their names always begin with a capital letter.
 - b. They return JSX markup.

Default Component: App.jsx

When a React project is created using Vite, App.jsx is created by default. It is the main component rendered in browser.

Rendering Flow

index.html → main.jsx → App.jsx

Example

Build basic react app that display “Hello World” in browser.

We can do this in two ways.

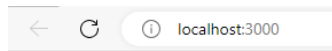
Method-1 Make changes in App.jsx file as shown below.

App.jsx

```
function App() {  
  return (  
    <div>  
      <h1>Hello World!</h1>  
    </div>  
  );  
}  
export default App;
```

This App.jsx file has been imported as a component in main.jsx file as we have shown above.

Code	Purpose
function App()	Creates React component
return()	Returns JSX/UI
<h1>Hello World!</h1>	Displays heading in browser
export default App	Exports component



Hello World!

This App.jsx file can also be considered as a component. As we are exporting it and importing in the file index.jsx. We can reuse this component just by importing it.

Method – 2 Create Your Own Component

Instead of modifying App.jsx, you can create your own component file and import in App.jsx or main.jsx. (Ideally import in App.jsx is better)

Create file named Pr1.jsx as a component and import this component in App.jsx file.

App.jsx

```
import Pr1 from './Pr1';
function App() {
  return (
    <div>
      <Pr1/> {/* This is a component. Component names must start with a capital letter. */}
    </div>
  );
}
export default App;
```

Another File: Pr1.jsx

```
import React from 'react'

function Pr1() {
  return (
    <div>
      <h1>Hello World! </h1>
    </div>
  )
}
export default Pr1
```

Note: Always follow 2nd method of importing component to App.jsx file. In this file just import component and call this component as shown in above example.

JSX (JavaScript XML)

- JSX is a syntax extension for JavaScript that lets you write HTML-like markup inside a JavaScript file.
- Each React component is a JavaScript function that may contain some markup that React renders into the browser.
- React components use a syntax extension called JSX to represent that markup.
- JSX looks a lot like HTML, but it is a bit stricter and can display dynamic information.

Simple HTML code

```
<h1>LJU students</h1>
<ul>
  <li>CSE</li>
  <li>IT</li>
  <li>CE</li>
</ul>
```

And if we want to put HTML code into our component:

```
function Ex1() {
  return (
    // If you copy and paste above code as it is here, it will not work and give an error:
    "Adjacent JSX elements must be wrapped in an enclosing tag. Use JSX fragment
    <>...</>" or <div>....</div> etc. and follow JSX Rules mentioned below
  )
}
export default Ex1
```

The Rules of JSX

1. Return a single root element

To return multiple elements from a component, wrap them with a single parent tag.

For example, you can use a <div>:

```
export default function App() {
  return (
    <div>
      <h1>LJU students</h1>
      <ul>
        <li>CSE</li>
        <li>IT</li>
      </ul>
    </div>
  )
}
```

If you don't want to add an extra `<div>` to your markup, you can write `<> and </>` instead of `<div>` tag.

This empty tag is called a **Fragment**. Fragments let you group things without leaving any trace in the browser HTML tree.

2. Close all the tags

JSX requires tags to be explicitly closed: self-closing tags like `<img>` must become `<img />`, and wrapping tags like `<li>oranges` must be written as `<li>oranges</li>`.

Examples:

```

<li>Apple</li>
<input type="submit" />
```

3. camelCase are required

JSX turns into JavaScript and attributes written in JSX become keys of JavaScript objects. In your own components, you will often want to read those attributes into variables. But JavaScript has limitations on variable names. For example, their names can't contain dashes or be reserved words like **class**.

This is why, in React, many HTML and SVG attributes are written in camelCase. For example, instead of **background-color** you use **backgroundColor**. Since **class** is a reserved word, in React you write **className** instead.

Examples:

```

<button onClick={fun}>Click</>
```

HTML	JSX	Reason
class	className	class is reserved keyword in JavaScript
onclick	onClick	JSX uses camelCase for events
onchange	onChange	JSX uses camelCase
readonly	readOnly	JSX uses camelCase

4. Passing expression

If you want to dynamically specify the src or alt text in img tag. You could use a value from JavaScript by replacing `" "` with `{ }`.

```
// import pic from "./img.png" also work to load image in <img>
function Avatar() {
  const n = "abc"
  const pic = 'img.png';
  const description = 'test image';
  return (
    <>
      <h1> Welcome {n} </h1>
      <img className="pic" src={pic} alt={description} />
    </>
  )
}
export default Avatar;
```

JSX Expression	React Understands As	Output
{n}	Evaluate variable n	lju
"{n}"	Text " + value of n + text "	"lju"
{"{n}"	JavaScript string "{n}"	{n}
{"n"	JavaScript string "n"	n
{n + " test"}	JavaScript expression	lju test
{n.toUpperCase()}	Execute JS method	LJU
{5 + 5}	JavaScript math expression	10
{[1,2,3]}	Array rendering	123
{[1,2,3].join(",")}	Array join method	1,2,3

Note: The **className="pic"**, which specifies and **"pic" CSS class name** that applies css to the image

src={pic} that reads the value of the JavaScript variable called **pic**. That's because **curly braces** let you work with JavaScript right there in your markup!

You can only use curly braces in two ways inside JSX:

As text directly inside a JSX tag: **<h1>{name}'s To Do List</h1>** works, **but <{tag}> Test's To Do List</{tag}>** will not work.

As attributes immediately following the = **sign: src={pic}** will read the avatar variable, **but src="{pic}"** will pass the string "{pic}".

5. Using “double curly braces”: CSS and other objects in JSX

In addition to strings, numbers, and other JavaScript expressions, you can even pass objects in JSX.

You may see this with inline CSS styles in JSX. React does not require you to use inline styles (CSS classes work great for most cases). But when you need an inline style, you pass an object to the style attribute:

Styling Rules

Rule	Example
Use double curly braces	style={{ }}
Use camelCase CSS	backgroundColor
CSS values are strings	"red"

```
function Subtraction() {
  return(
    <div>
      <h1 style={{backgroundColor:'red',color:'#fff'}}>Subtraction : {7-4}</h1>
    </div>
  );
}
```

OR

```
function Subtraction() {
  var mystyle = {backgroundColor:'red',color:'#fff'};
  return(
    <div> <h1 style={mystyle}>Subtraction : {7-4}</h1> </div>
  )
}
```

6. JSX Comments

To write comments in React (JSX), we need to wrap **them in curly braces**.

```
Function comment() {
  Return(
    { /* this works */ }
  )
}
```

The curly braces tell the JSX parser to parse the code inside as JavaScript, and not a string.

Since the contents inside are parsed as JavaScript, this enables us to also do multi-line or single-line comments:

```
function comment(){
  return (
    <>
    {
```

```

    /*
      mult-line
      test
    */
  }
  {
    // single-line test
  }
</>
))

```

In the case of a single-line comment, You cannot have the ending bracket in the same line, because that will break everything.

Example 1

Build a basic React application using function component with the following requirements:

1. **Create one h1 heading displaying welcome text in:**
 - **Blue color**
 - **Italic style**
 - **Font size 25px (using inline CSS)**
2. **Display one paragraph using external CSS (className).**
3. **Create an ordered list of 3 fruits using alphabetical numbering.**
4. **Display the current date and current time using JavaScript.**
5. **Display one image from the assets folder.**
6. **Display one image from the public folder.**
7. **Store application name in a variable and display it inside heading using JSX expression.**

Myapp.jsx

```

import './My.css'
import img1 from './assets/hero.png'
function Myapp() {
  const name = "React"
  const date=new Date().toLocaleDateString()
  const time=new Date().toLocaleTimeString()
  return (
    <>
      {/* Inline CSS */}
      <h1 style={{
        color: "blue",
        fontStyle: "italic",
        fontSize: "25px"
      }}>
        Welcome to {name}
      </h1>
    </>
  )
}

```

```

    </h1>

    {/* External CSS using className */}
    <p className="para">
      JSX allows writing HTML-like code inside JavaScript.
    </p>

    {/* Ordered List */}
    <h3>List of Fruits</h3>
    <ol type="A">
      <li>Apple</li>
      <li>Mango</li>
      <li>Kiwi</li>
    </ol>

    <h6>Current Date: {date}</h6>
    <h6>Current Time: {time}</h6>
    {/* Image from assets folder */}
    <img src={img1} alt="fruit" width="200" />

    {/* Image from public folder */}
    

  </>
)
}
export default Myapp

```

My.css

```

.para {
  color: green;
  font-size: 20px;
}

```

Note: Import `Myapp.jsx` component in `App.jsx`

Note: Styling info must be written inside two sets of curly braces `{{}}`. If dashed property name is used like background-color, font-size, then we have to use camel case names of properties like backgroundColor, fontSize.

****new Date().toLocaleDateString():** The toLocaleDateString() method returns the date (not the time) of a date object as a string, using locale conventions.

****new Date().toLocaleTimeString():** The toLocaleTimeString() method returns the time portion of a date object as a string, using locale conventions.

****Alternatively components can be created by using ES6 arrow function.**

Map and Filter

Purpose of map() in React

- map() is used to loop through array elements.
- It helps to display data dynamically in React.
- It transforms each array item into JSX.
- It returns a new array.
- Mostly used for rendering lists.

Purpose of filter() in React

- filter() is used to remove unwanted data.
- It checks condition on every array element.
- It returns only matching elements.
- It creates a filtered array.
- Mostly used before map().

List and Keys

- ✓ key is used to give a unique identity to each element in a list.
- ✓ React uses key to identify which item is:
 - a. added
 - b. updated
 - c. deleted
- ✓ It helps React update the UI efficiently.
- ✓ key improves performance while rendering lists.
- ✓ It is mainly used with map() method.
- ✓ If **lists** do not include the **key** attribute, then it will give below warning. The warning says that each of the list items in our unordered list should have a unique key.

**Warning: Each child in an array or iterator
should have a unique "key" prop**

Keys are used in React to identify which items in the list are changed, updated, or deleted. In other words, we can say that keys are used to give an identity to the elements in the lists. It is recommended to use a string as a key that uniquely identifies the items in the list.

Use of key is given in below example. Without adding it you will get a warning as mentioned above.

[Refer Annexure -II for more detail]

Example-1: We have an array of numbers and we want to multiply each of these numbers by 5. Create react app to display these multiplied numbers **using map method**.

Map2.jsx

```
import React from 'react';

function Map2() {
  let arr = [2, 4, 6, 3, 10, 12]
```

```

    return (
      <div>
        <h1>Multiplication of numbers are as under: </h1>
        {
          arr.map((value)=>{
            return <h2>{value} * 5 = {value * 5}</h2>
          })
        }
      </div>
    )
  }
}
export default Map2

```

App.jsx

import Map2 from "./Map2"; and call component in App.jsx

Output:

Multiplication of numbers are as under:

```

2 * 5 = 10
4 * 5 = 20
6 * 5 = 30
3 * 5 = 15
10 * 5 = 50
12 * 5 = 60

```

Example 2: Write React code to filter out the numbers greater 6 using map/filter method.

Check difference in output using different methods.

Using Only filter method

```

const ArrayMap_condition = () => {
  const arr1=[1,2,3,4,5,6,7,8,9]
  return (
    <div>
      <h1>using map/filter function</h1>
      {
        arr1.filter((value)=>value <=6){
          return <h1>array values= {value}</h1>
        }
      })}
    </div>
  ) }
export default ArrayMap_condition

```

Output using filter function:

```
using map/filter function
123456
```

Filtering First: arr1.filter filters the array to only include values less than or equal to 6 and displayed filtered values to **without** applying <h1> element.(returns only values)

This is not valid because:

- filter() expects a boolean return value (true/false), not JSX.
- Returning a <h1> here doesn't make logical sense to .filter().

Or

Using Only map method

```
const ArrayMap_condition = () => {
  const arr1=[1,2,3,4,5,6,7,8,9]
  return (
    <div>
      <h1>using map/filter function</h1>
      {
        arr1.map((value)=>{
          if(value <=6){
            return <h1>array values= {value}</h1>
          }
        })
      }
    </div>
  ) }
export default ArrayMap_condition
```

Output using map function:

```
using map/filter function
array values= 1
array values= 2
array values= 3
array values= 4
array values= 5
array values= 6
```

- Cons: The map method will still iterate over all elements, but only those that meet the condition will render something. **Others will effectively render undefined.**

Optimized Approach: filter first, then map() to JSX.

Using filter and map methods

```
const ArrayMap_condition = () => {  
  const arr1=[1,2,3,4,5,6,7,8,9]  
  return (  
    <div>  
      <h1>using map/filter function</h1>  
      {  
arr1.filter((value)=>value <=6).map((value)=>{ return <h1>array values= {value}</h1>})  
      }  
    </div>  
  ) }  
export default ArrayMap_condition
```

Output using map and filter function:

```
using map/filter function  
array values= 1  
array values= 2  
array values= 3  
array values= 4  
array values= 5  
array values= 6
```

- Filtering First: arr1.filter((value) => value <= 6) filters the array to only include values less than or equal to 6.
- Mapping with Keys: map((value)=>{ return <h1>array values= {value}</h1>}) maps the filtered values to <h1> elements.

Best Practice:

To avoid unnecessary **undefined** values (even if they're harmless here) using map method with conditions (observe Console.log(t)),
prefer:
arr.filter(val => val > 3).map(val => <h2>{val}</h2>)
It keeps your logic **cleaner, predictable, and debug-friendly**.

Example

Create a React component that contains an array of student objects with:

- id
- name
- standard (std)

Perform the following tasks using filter() and map():

1. Display only students whose standard is 5.
2. Increase their standard by 1.
3. Display student name and updated standard.

ProductList.jsx

```
function StudentList() {
  const students = [{ id: 1, name: "abc", std: 4},
                    { id: 2, name: "xyz", std: 6},
                    { id: 3, name: "pqr", std: 5} ];

  return (
    <div>
      <h1>Student Details</h1>

      {
        students
        .filter((student) => student.std === 5)
        .map((student) => (
          <div key={student.id}>
            <h2>Student Name : {student.name.toUpperCase()}</h2>
            <h3>Updated Standard : {student.std + 1}</h3>

            <hr />
          </div>
        ))
      }
    </div>
  );
}
export default StudentList;
```

**Alternate Way

```
.map((student,index) =>{
  return (
    <div key={index}>{student.name}
    </div>
  )
})
```

React Props

- Props stand for "Properties." It is an object which stores the value of attributes of a tag and work similar to the HTML attributes.
- React components use props to communicate with each other. Each component can pass some information to other components by giving them props.
- Props are similar to function arguments. Props are passed to the component in the same way as arguments passed in a function.
- Props are used to pass data from one component to another component.
- Props work similar to function arguments in JavaScript.
- Props help React components communicate with each other.
- Props are read-only (immutable), meaning we should not modify them directly.

Why We Use Props

Props are used to:

- Pass data between components
- Make components reusable
- Display dynamic content
- Communicate from parent component to child component

Syntax of Props

Passing Props

```
<ComponentName propertyName="value" />
```

Reading Props

```
function ComponentName(props) {  
  return <h1>{props.propertyName}</h1>  
}
```

Example 1:

Product Name and its price are declared as props in Pr2.jsx file and the value of props passed in app.jsx file. A variable can also be passed as props value.

In App.jsx mention property for passing to specific file (here inside `<pr2 property = value >`

In Pr2.jsx call the value using: `props.property` (props is parameter pass in function)

Step 1: Pass props to the component in app.jsx file:

```
import Pr2 from "./Pr2";  
  
function App() {  
  const xyz="Computer";  
  return (  
    <div>
```

```
<Pr2 Name={xyz} Price="70000" />
<Pr2 Name="Mobile" Price="20000" />
</div> ) }
export default App;
```

Step 2: Read props inside the component In Pr2.jsx file

```
import React from 'react'
function Pr2 (props){
  return(
    <div>
      <h1>{props.Name} : {props.Price}</h1>
    </div> )}
export default Pr2
```

Output:

```
Computer : 70000
Mobile : 20000
```

Example 2

Create a React application using props to display student details.

Requirements:

1. Create a parent component Prop1.jsx.
2. Pass the following data to child component Prop2.jsx using props:
 - student name
 - roll number
 - marks
3. In child component:
 - display student details
 - increase marks by 1 and display updated marks

Understand the difference between:
marks={20} marks="22" from example

Step 1: Pass props to the component

Prop1.jsx

```
import Prop2 from "./Prop2";

function Prop1 () {
  var n = "ABC";
  return (
    <div>
```

```
    <Prop2 name={n} rollnum="101" marks={20} />
    <Prop2 name="DEF" rollnum="102" marks={16} />
    <Prop2 name="GHI" rollnum="103" marks="22" />
  </div>
);
}
export default Prop1;
```

Step 2: Read props inside the component

Prop2.jsx

```
function Prop2(props){
  return(
    <div>
      <ul>
        <li>{props.name}</li>
        <li>{props.rollnum}</li>
        <li>{props.marks + 1}</li>
      </ul>
    </div>
  );
}
export default Prop2;
```

Output:

```
ABC
101
21

DEF
102
17

GHI
103
221
```

Difference Between "" and {}

Syntax	Type
"22"	String
{22}	Number / JavaScript value

Example

Create a React application using props to pass an object from one component to another component.

Requirements:

1. Create a parent component Example1.jsx.
2. Create an object named Details containing:
 - Student_name
 - University_name
3. Pass the Details object to child component Example.jsx using props.
4. In child component:
 - receive the object using props
 - display student name and university name in browser

Example1.jsx

```
import Example from './Example';
function Example1 () {
const Details = {Student_name: "abc", University_name: "LJU"};
return (
<div>
<Example data ={ Details }/>
</div>
) }
export default Example1
```

Example.jsx

```
function Example(props) {
return(
<h2> My name is {props.data.Student_name}.
I am a student of { props.data.University_name } University !</h2>
) }
export default Example
```

Output:

My name is abc. I am a student of LJU University !

Example

Build a React app that displays a list of products using props.

- Create a **ProductCard** component to show product details like title, price, rating, and image.
- Use a **ProductList** component to store product data and display multiple **ProductCard** components using `.map()`.

ProductList.jsx

```
import ProductCard from './ProductCard';
import img1 from "./img1.png"
import img2 from "./img2.png"

function ProductList() {
  const products = [ { title: "iPhone 17 Pro", price: "99999", rating: 4.8, image: img1 },
    { title: "Samsung Galaxy S24", price: "60000", rating: 4.5, image: img2 } ];

  return (
    <div>
      <h1>Our Products</h1>
      <ProductCard productList={products} />
    </div>
  );
}

export default ProductList;
```

ProductCard.jsx

```
function ProductCard(props) {
  return (
    <>
      {
        props.productList.map((product, index) => (
          <div>
            <img src={product.image} alt={product.title} width="100%" />
            <h2>{product.title}</h2>
            <p>Price: {product.price}</p>
            <p>Rating: {product.rating}</p>
          </div>
        ))
      }
    </>
  );
}

export default ProductCard;
```

React Events

To handle events with React elements is very similar to handle events with DOM events. There are some syntax differences as below:

React events are named using camelCase, rather than lowercase as we used to do in HTML.

In HTML:

```
<button onclick="demofunction ()">
  LJ University
</button>
```

In React:

```
<button onClick = { demoFunction }>
  LJ University
</button>
```

onClick event

Write react script to display alert box with text “Welcome to LJU” by clicking on button.

Event1.jsx

```
import React from 'react';
function Event1() {
  const mystyle = {
    color : "white", backgroundColor: "#000000",
    padding: "10px 20px", margin: "200px"
  };

  const handleClick = () => {    //(Note that here ES 6 Arrow Function is used instead of js function)
    alert ('Welcome to LJU');
  }

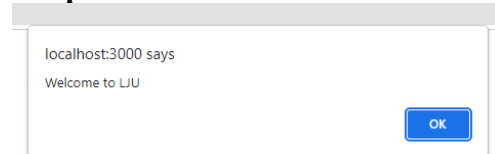
  return (
    <div>
      <center>
        <button style = {mystyle} onClick={handleClick}>
          /*<button onClick={handleClick}>*/
          Click me
        </button>
      </center>
    </div>
  );
}
```

```
}  
export default Event1;
```

App.jsx

```
import Event1 from "./Event1"  
function App() {  
  return (  
    <div>  
      <Event1/>  
    </div>  
  );  
}  
export default App;
```

Output:



Click me

Passing Arguments

To pass an argument to an event handler, use an arrow function.

Example:

Send "Goal!" as a parameter to the shoot function, using arrow function:

```
function Football() {  
  const shoot = (a) => {  
    alert(a);  
  }  
  return (  
    <button onClick={() => shoot("Goal!")}>Take the shot!</button>  
  );  
}
```

onDoubleClick event

Write react js script to display alert box with text “welcome to lju” only on double click.

Event3.jsx

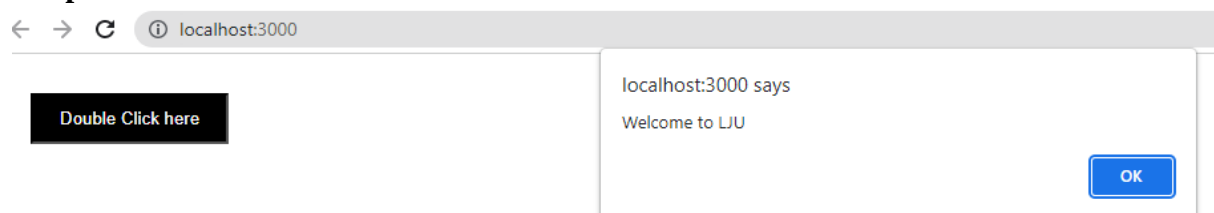
```
import React from 'react';
function Event3() {
  const mystyle = { color : "white",   backgroundColor : "#000000",
    padding: "10px 20px",   margin:"30px"  };

  const doubleClickHandler = () => {
    alert("Welcome to LJU");
  }
  return (
    <>
    <button style={mystyle} onDoubleClick = {doubleClickHandler}>Double Click
here</button>
    </>
  );
}
export default Event3;
```

App.jsx

```
import Event3 from "./Event3"
function App() {
  return (
    <div>
      <Event3/>
    </div>
  );
}
export default App;
```

Output:



onChange event

An onChange event handler returns a Synthetic Event object which contains useful meta data such as the target input's id, name, and current value.

We can access the target input's value inside of the handleChange by accessing event.target.value.

event.target gives you the element that triggered the event. So, **event.target.value** retrieves the value of that element (an input field, in your example). In React, events are SyntheticEvent, a wrapper around the browser's native event. It has the same interface as the browser's native event, including stopPropagation() and preventDefault(), except the events work identically across all browsers.

- Write react js script to display values in console while changing it in text box.

Event2.jsx

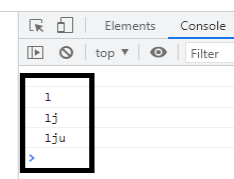
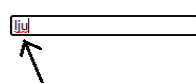
```
import React from 'react';
function Event2() {
  function handleChange(event) {
    console.log (event.target.value);
  }
  return (
    <input type="text" name="firstName" onChange={handleChange} />
  ) }
export default Event2;
```

App.jsx

```
import Event2 from "./Event2"
function App() {
  return (
    <div>
      <Event2/>
    </div>
  ) }
export default App;
```

Output:

OnChange Example



event.target gives the element that triggered the event.

So, event.target.value retrieves the value of that element (an input field, in above example).

onSubmit event

Another difference is that we cannot **return false** to prevent default behavior in React. We must call **preventDefault** explicitly.

**** Use the `preventDefault()` method on the event object to prevent a page refresh on form submit in React. It prevents the browser from issuing the default action which in the case of a form submission is to refresh the page.**

preventDefault is used to prevent the default form behavior of submitting, reloading etc..

In HTML(one can write this way)

```
<form onsubmit="console.log (' You clicked submit. '); return false">
  <button type="submit">Submit</button>
</form>
```

Write react js script to display onSubmit event.

In Form.jsx

```
import React from 'react';

function Form () {
  function handleSubmit (e) {
    e.preventDefault ();
    alert (' You clicked submit. ');
  }
  return (
    <form onSubmit = {handleSubmit}>
      <input type="text"/>Enter name here
      <button type="submit">Submit</button>
    </form>
  );
}
export default Form;
```

In App.jsx file:

```
import Form from "./Form";
function App() {
  return (
    <div>
      <Form/>
    </div>
```

```
) }
export default App
```

Example to understand concept of function calling without () [ref* only]

```
function App() {
  const handleClick = () => {
    alert("Welcome to LJU") };
  return (
    <div>
      <center>
        <button onClick={handleClick}>Click me</button>
      </center>
    </div> )}
```

Output:

On page load, the alert pops up: "Welcome to LJU"

But when you click the button, nothing happens because the onClick handler is now undefined.

Explanation:

The function handleClick() is being called immediately inside onClick={handleClick()} instead of passing a reference. So, as soon as the component renders, it will call the function handleClick(). An alert box appears with the message "Welcome to LJU"

Example: (Without Hook. Using Events only)

Create a component to perform the tasks as described below:

1. Add a text field and a submit button.
 - While changing the value in the text field, display it below the form.
 - Display this text field value in an alert box upon submitting it.
2. Add a button to perform click and double-click event tasks.
 - On click event, display message in h3 tag "You clicked once".
 - On double-click event, display message in h3 tag "You clicked twice".
 - Display these message should be displayed below the button.

```
function Map1() {

  function handleSubmit (e) {
    e.preventDefault ();
    alert (document.getElementById('uname').value);
  }
}
```



```
function handleClick(){
  document.getElementById('test1').innerHTML = "You clicked once"
}

function handledoubleclick(){
  document.getElementById('test1').innerHTML = "You clicked twice"
}

function handleChange(event) {
  document.getElementById('test').innerHTML =event.target.value;
}

return (
  <div>
    <form onSubmit = {handleSubmit}>
      <input type="text" id="uname" onChange={handleChange}></input>
      <input type="submit"/>
    </form>

    <h1 id="test">On change event</h1>

    <button style={{backgroundColor:'black',padding:"20px",color:"white"}}
onClick={handleclick} onDoubleClick={handledoubleclick}>Click</button>
    <h1 id="test1">Click/DoubleClick event</h1>
  </div>
)
}

export default Map1
```

React Routing

Create React App doesn't include page routing. React Router is the most popular solution for page routing.

Routing is a process in which a user is directed to different pages based on their action or request. ReactJS Router is mainly used for developing Single Page Web Applications. React Router is used to define multiple routes in the application. When a user types a specific URL into the browser, and if this URL path matches any 'route' inside the router file, the user will be redirected to that particular route.

React Router is a standard library system built on top of the React and used to create routing in the React application using React Router Package. It provides the synchronous URL on the browser with data that will be displayed on the web page. It maintains the standard structure and behavior of the application and mainly used for developing single page web applications.

Purpose of React Router

- Navigate between pages
- Create SPA (Single Page Application)
- Change URL dynamically
- Render components based on URL path

React contains three different packages for routing. These are:

1. **react-router:** It provides the core routing components and functions for the React Router applications.
2. **react-router-native:** It is used for mobile applications.
3. **react-router-dom:** It is used for web applications design.

Routing structure

1. **Store “BrowserRouter as Router”:** A `<router>` that uses the hash part of the URL to keep your UI in sync with the URL. It is the parent component that is used to store all of the other components.

Syntax

<pre><BrowserRouter> App Components </BrowserRouter></pre>
--

2. **Routes:** To read a single component, wrap all routes inside the Routes component. Switch groups across multiple routes, iterate over them and find the first one that matches the path. Thus, the corresponding component of the path is rendered. An application can have multiple `<Routes>`. Routes are chosen based on the best match instead of being traversed in order.

3. **Route:** The route component will help us establish the link between the component's UI and the URL. To include the route in the application, add the below code to your app.js.

It is used to define and render component based on the specified path. It will accept components and render to define what should be rendered.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/contact" element={<Contact />} />
</Routes>
```

4. **Link:** Link component is used to create links to different routes and implement navigation around the application. It works like HTML anchor tag.

```
<Link to="/home">Home</Link>
```

Note: Do not use anchor tags instead of <Link> components because using anchor tags would not allow applications to remain Single Page Application (SPA). HTML anchor tag would trigger a page reload or page refresh when clicked.

It is not possible to install react-router directly in your application. To use react routing, first, you need to install react-router-dom modules in your application. The below command is used to install react router dom.

Add React Router

To add React Router in your application, run below command in the terminal from the root directory of the application:

```
npm i react-router-dom
```

Routing Flow

URL changes



React Router checks matching path



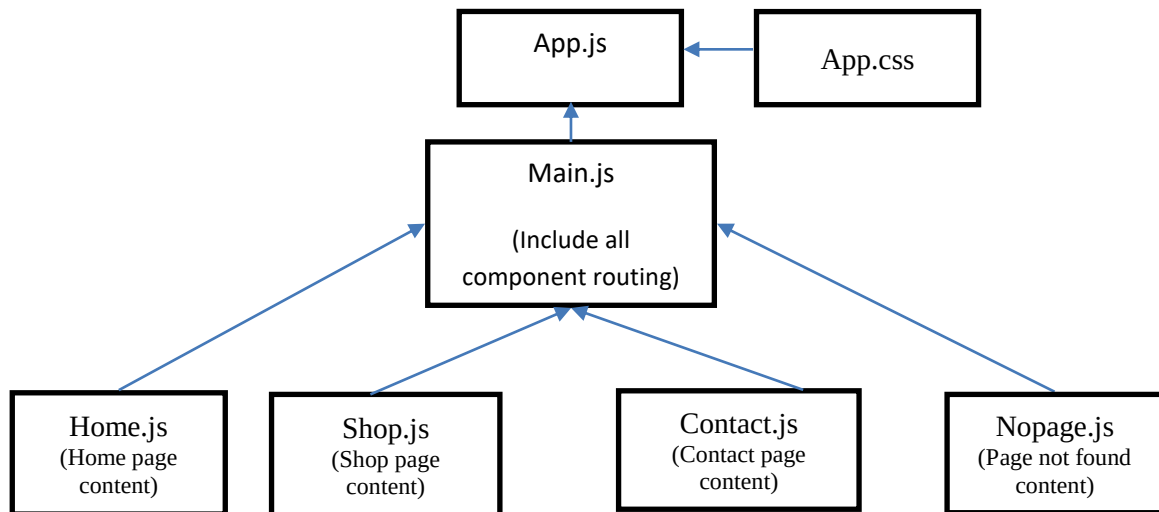
Matching component renders

Folder Structure

To create an application with multiple page routes, let's first start with the file structure. Within the **src** folder, we'll create a folder named **components** with several files:

src\components\:

- Home.jsx
- Shop.jsx
- Contact.jsx
- Nopage.jsx

**Example:**

Create react app to perform tasks as asked.

- First create files as asked below in component folder
 1. **Home.jsx** - for the home page content
 2. **Shop.jsx** - for the shop page content
 3. **Contact.jsx** - for the contact page content
 4. **Nopage.jsx** – for the page other than mentioned links
- Create **Main.jsx** file which contains Links for Home, Shop and Product page. Also, add functionality of page routing.
- Finally call Main.js in App.js.

App.jsx

```
import './App.css';
import Main from "./components/Main.jsx";

function App() {
  return (
    <div>
      <Main/>
    </div>
  );
}
export default App;
```

(Below all files are inside components folder in src folder.)

components /Main.jsx

```
import React from 'react';
import {BrowserRouter as Router,Route,Routes,Link} from "react-router-dom";
import Home from './Home';
import Contact from './Contact'
import Shop from './Shop'
import Nopage from './Nopage';
function Main() {
return (
<div >

  <Router>
    <div className='main-route'>
      <ul>
        <li><Link to="/">Home</Link></li>
        <li><Link to="/shop">Shop</Link></li>
        <li><Link to="/contact">Contact</Link></li>
      </ul>
    </div>
    <Routes>
      <Route path="/" element={ <Home/> }/>
      <Route path="Contact" element={ <Contact/> }/>
      <Route path="Shop" element={ <Shop/> }/>
      <Route path="*" element={ <Nopage/> }/>
    </Routes>
  </Router>

</div>
);
}
export default Main
```

Setting the path to * will act as a catch all undefined URLs and display 404 error page.

components/Home.jsx

```
import React from 'react'
function Home(){
return (
<div>
<h1>Home page</h1>
</div>
)}
export default Home
```

components /Shop.jsx

```
import React from 'react'
function Shop(){
return (
<div>
<h1>Shop page</h1>
```

```
</div>
)
}
export default Shop
```

components /Contact.jsx

```
import React from 'react'
function Contact() {
  return (
    <div>
    <h1>Contact Detail</h1>
    </div>
  )
}
export default Contact
```

components /Nopage.jsx

```
import React from 'react'
function Nopage() {
  return (
    <div>
    <h1>404 Page not Found</h1>

    </div>
  )
}
export default Nopage
```

App.css (It is Not compulsory to add this file in this example. Added css for the reference only)

```
.main-route ul {
  list-style-type: none;
  margin: 0;
  padding: 0;
  overflow: hidden;
  background-color: #000;
  margin-bottom: 50px;
}
.main-route li {
  float: left;
}

.main-route li a {
  display: block;
  color: white;
  text-align: center;
  padding: 20px;
  text-decoration: none;
}
.main-route h1{ color: red; text-align: center;}
```

Example 2

Create a React app to perform the following tasks using functional components:

Implement the following components in your React application:

- Main.jsx to set up the router and define the routes.
 - Home.jsx for the Home page.
 - Product.jsx for the Product page.
1. **Create a React Router:**
 - Include two routes: Home and Product. Implement navigation between these routes.
 2. **Create the following routes and components:**
 - When a user clicks on the Home page link, it should navigate to the Home page and display "Welcome to LJU" within an <h1> heading with blue color. Also, include link to product page.
 - A **Product** page that displays three products' information (name, price, and image) using props. When a user clicks on the Product page link, it should navigate to the Product page and display three products' information name, price and product image using props.

Main.jsx

```
import {BrowserRouter as Router,Route,Routes,Link} from "react-router-dom";
import Home from './Home';
import Product from './Product';
import Nopage from './Nopage';
import img1 from "./img1.jpg"
import img2 from "./img2.jpg"
import img3 from "./img3.png"
function Main() {
  const products=[{name:"p1",price:20000, pic:img1},{name:"p2",price:14000,
pic:img2},{name:"p3",price:40000, pic:img3}]
  return (
    <div>
      <Router baseName="/calendar">
        <div className='main-route'>
          <ul>
            <li><Link to="/">Home</Link></li>
            <li><Link to="/product">Product</Link></li>
          </ul>
        </div>
        <Routes>
          <Route path="/" element={<Home/>}/>
          <Route path="/product" element={<Product info={products}/>}/>
          <Route path="*" element={<Nopage/>}/>
        </Routes>
      </Router>
    </div>
```

```
);  
}  
export default Main
```

Home.jsx

```
function Home(){  
  return (  
    <div>  
      <h1 style={{color:"blue"}}>Home page</h1>  
      <a href="/product">Product</a>  
    </div>  
  )  
}  
export default Home
```

Product.jsx

```
const Product = (Props)=>{  
  return(  
    <div>  
      {  
        Props.info.map((p)=>{  
          return(  
            <div>  
              <img src={p.pic} alt="product" height={200} width={200}/>  
              <h1>{p.name}</h1>  
              <h1>{p.price}</h1>  
            </div>  
          )  
        })  
      }  
      <h1>{Props.Name} : {Props.Price}</h1>  
    </div>  
  );}  
export default Product;
```

Nopage.jsx

```
function Nopage() {  
  return (  
    <div>  
      <h1>404 Page not Found</h1>  
    </div>  
  )}  
export default Nopage
```


Miscellaneous Tasks

Task 1: Make a JSX file named ex1.jsx, which contains two functions: one for addition and one for subtraction.

Pass these functions as component inside another function(ex1) in same file and display addition and subtraction in browser.

Ex1.jsx

```
function Add() {
  return (
    <h1> Addition: {5+5} </h1>
  )
};
function Subtraction() {
  return(
    <div>
      <h1>Subtraction : {7-4}</h1>
    </div>
  )
};
function Ex1() {
  return(
    <div>
      <Add /> //Must be start with capital letter as we are using it as a component
      <Subtraction /> //Must be start with capital letter as we are using it as a component
    </div>
  )}
export default Ex1;
```

App.jsx

```
import Ex1 from "./Ex1";
function App() {
  return (
    <div>
      <Ex1/>
    </div>
  );
}
export default App;
```

Task 2: Create React app to pass color(red), background color(yellow), font size(25px) and font style(italic) as properties to component and apply css to "Lj Students" text written in p tag. (Props)

Example.jsx

```
import PEx1 from "./Example1";
function Example() {
  const cssdata = { color: "red", backgroundColor: "yellow", fontSize:"25px" };
  return (
    <div>
      <PEx1 css={ cssdata }/>
    </div>
  ) }
export default Example
```

ex1.jsx

```
const Ex1 = (props)=>{
  var cl = { color:props.css.color, backgroundColor:props.css.backgroundColor,
  fontSize:props.css.fontSize
  }

  return(
    <p style={cl}>LJ students</p>
  );}
export default Ex1;
```

Task 3: Create ReactJS app to pass student name, roll number, t1marks and t2marks of 2 students to component and read the information and display in table format using Props.

In App.jsx

```
import React from 'react';

import StudentTable from './B229';

function App() {
  const students = [{name: 'Student 1', rollNumber: '001', t1marks: 19, t2marks: 25},
  {name: 'Student 2', rollNumber: '002', t1marks: 20, t2marks: 24},,];

  return (
    <div className="App">
      <h1>Student Information</h1>
      <StudentTable students={students} />
    </div>
```

```
) }  
export default App;
```

In Data.jsx

```
import React from 'react';  
  
const Data = ({ students }) => {  
  
  return (  
    <table border="1">  
      <thead>  
        <tr>  
          <th>Name</th>  
          <th>Roll Number</th>  
          <th>Test 1 Marks</th>  
          <th>Test 2 Marks</th>  
        </tr>  
      </thead>  
      <tbody>  
        {students.map((student) => (  
  
          <tr>  
            <td>{student.name}</td>  
            <td>{student.rollNumber}</td>  
            <td>{student.t1marks}</td>  
            <td>{student.t2marks}</td>  
          </tr>  
        ))}  
  
      </tbody>  
    </table>  
  )};  
  
export default Data;
```

Annexure – I

Once a react app gets created, The **folder structure** looks as below.

Rendering Flow

index.html → main.jsx → App.jsx

- index.html contains root div
- main.jsx connects React with browser
- App.jsx displays actual UI

Folder Structure

```

myapp
├── node_modules
├── public
│   ├── favicon.svg
│   └── icons.svg
├── src
│   ├── assets
│   ├── App.css
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── .gitignore
├── eslint.config.js
├── index.html
├── package-lock.json
├── package.json
├── README.md
└── vite.config.js
  
```

File / Folder	What it Contains	Purpose / Use	Important Notes
node_modules/	All installed npm packages	React, Vite and other libraries are stored here	Created automatically after npm install. Very large folder. Never edit manually.
public/	Static files	Stores files that should be directly accessible in browser	Files are not processed by Vite. Used for favicon, PDFs, static images, etc.
src/	Main React code	Contains all components, CSS, logic and pages	Most project development happens here.
src/assets/	Images, icons, fonts, videos	Used to store component-related media files	Best place for images used inside React components.
src/App.jsx	Main React component	First component displayed in browser	Main UI design is usually written here.
src/main.jsx	Entry point file	Starts React app and connects with index.html	Flow: index.html → main.jsx → App.jsx
src/App.css	CSS for App component	Styles only App component UI	Used for component-specific styling.
src/index.css	Global CSS	Applies styles to whole application	Used for body styling, fonts, reset CSS, etc.
.gitignore	Ignored files list	Prevents unnecessary files from uploading to GitHub	Usually ignores node_modules, build files, environment files.
eslint.config.js	ESLint rules	Checks coding mistakes and code quality	Helps maintain clean and error-free code.

index.html	Main HTML page	Base HTML file loaded in browser	Contains <code><div id="root"></div></code> where React app renders.
package.json	Project configuration	Stores dependencies, scripts and project details	Contains commands like <code>npm run dev</code> , installed package names, versions.
package-lock.json	Exact dependency versions	Locks package versions for consistency	Automatically generated by npm.
README.md	Documentation file	Contains setup instructions and project notes	Written in Markdown format.
vite.config.js	Vite configuration	Used to customize Vite settings	Used for custom port, aliases, plugins, etc.

index.html

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>myapp</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

It is the main HTML file of the React application.
When the application starts, this file loads first in the browser.

Code	Purpose	Explanation
<code><!doctype html></code>	Defines HTML version	Tells browser to use HTML5.
<code><html lang="en"></code>	Root HTML tag	<code>lang="en"</code> specifies webpage language as English.
<code><head></code>	Metadata section	Contains title, favicon, charset and other settings.
<code><meta charset="UTF-8" /></code>	Character encoding	Supports special characters, symbols and multiple languages.
<code><link rel="icon" type="image/svg+xml" href="/favicon.svg" /></code>	Website icon	Sets favicon shown in browser tab. File comes from <code>public/</code> folder.
<code><meta name="viewport" content="width=device-width, initial-scale=1.0" /></code>	Responsive design	Makes website responsive on mobile, tablet and desktop

<code>scale=1.0" /></code>		screens.
<code><title>myapp</title></code>	Browser tab title	Displays myapp on browser tab.
<code><body></code>	Visible webpage section	Contains content displayed in browser.
<code><div id="root"></div></code>	Root container	React app renders inside this div.
<code><script type="module" src="/src/main.jsx"></script></code>	Loads React app	Starts application from main.jsx. type="module" enables modern JavaScript modules.

main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Code	Purpose	Explanation
<code>import { StrictMode } from 'react'</code>	Imports StrictMode	Helps detect warnings and potential problems during development.
<code>import { createRoot } from 'react-dom/client'</code>	Imports createRoot	Used to render React app into browser DOM.
<code>import './index.css'</code>	Imports global CSS	Applies CSS styles to entire application.
<code>import App from './App.jsx'</code>	Imports App component	Loads the main React component.
<code>document.getElementById('root')</code>	Selects root div	Finds <code><div id="root"></div></code> from index.html.
<code>createRoot(...)</code>	Creates React root	Initializes React rendering system.
<code>.render()</code>	Renders React app	Displays React components in browser.
<code><StrictMode></code>	Development checking tool	Shows warnings for unsafe or outdated React code.
<code><App /></code>	Main component	Main UI component rendered inside root div.

App.jsx

Code	Purpose	Explanation
import { useState } from 'react'	Imports Hook	useState is used to create and manage state in React.
import reactLogo from './assets/react.svg'	Imports image	Loads React logo from assets folder.
import viteLogo from './assets/vite.svg'	Imports image	Loads Vite logo from assets folder.
import heroImg from './assets/hero.png'	Imports image	Loads hero image from assets folder.
import './App.css'	Imports CSS	Applies styles for App.jsx.
function App()	React component	Main functional component of application.
const [count, setCount] = useState(0)	State creation	Creates state variable count with initial value 0.
Count	State variable	Stores current counter value.
setCount()	State update function	Updates count value.
return()	Returns JSX	Displays UI elements in browser.
<> </>	Fragment	Groups multiple elements without adding extra HTML tags.
<section id="center">	Section container	Main content section of page.
	Displays image	Shows imported image in browser.
className="hero"	CSS class	Applies CSS styling. React uses className instead of class.
<h1>Get started</h1>	Heading	Displays main heading.
<code>	Code formatting	Displays text in code style.
<button>	Button element	Creates clickable button.
onClick={() => setCount(...)}	Click event	Runs function when button is clicked.
setCount((count) => count + 1)	Updates state	Increases counter value by 1.
Count is {count}	Dynamic value	Displays updated counter value using JSX expression.
	Hyperlink	Opens Vite documentation website.
target="_blank"	Open in new tab	Opens link in new browser tab.
<svg>	SVG icon	Displays vector icons.
export default App	Export component	Makes App component available for use in main.jsx.

Note:

React components can be created using both .js and .jsx extensions. .jsx is recommended for component files because it clearly indicates the file contains JSX (React UI code)

Extension	Usage
.jsx	React components
.js	JavaScript logic/helper files

Always Component Names Start with Capital Letters

- React uses capitalization to identify components.
- Capital letters help React distinguish components from normal HTML elements.
- It is recommended to create component file names starting with capital letters to avoid confusion.

Name	React Treatment
Header	React component
header	HTML Tag

Correct:

```
function Header() {}
```

Wrong:

```
function header() {}
```

package.json

package.json is the main configuration file of a React project.

It contains:

- project information
- dependencies/packages
- scripts/commands
- package versions

Example:

```
{
  "name": "myapp",
  "version": "0.0.0",
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  }
}
```

Important Fields in package.json

Field	Purpose
name	Project name
version	Project version
scripts	Commands like npm run dev
dependencies	Installed packages
devDependencies	Development packages

React Rendering

React renders UI using:

- `createRoot()`
- `render()`

Example

```
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
createRoot(document.getElementById('root')).render(
  <App />
)
```

warnings.

React render

React renders HTML to the web page by using a function called **`createRoot()`** and its method **`render()`**.

The `createRoot` Function

The `createRoot()` function takes one argument, an HTML element.

The purpose of the function is to define the HTML element where a React component should be displayed.

The `render` Method

The `render()` method is then called to define the React component that should be rendered.

When building web applications in React, you use two packages—**react** and **react-dom**.

The **react** package holds the react source for components, state, props and all the code that is react.

The **react-dom** package as the name implies is the glue between React and the DOM. Often, you will only use it for one single thing: mounting your application to the `index.html` file with **`ReactDOM.render()`**.

React

Understands JSX (`<h1>`, `<button>`, etc.)

Manages component logic (`useState`, rendering, updating)

React-dom

Finds `document.getElementById('root')`

Injects React's virtual DOM into the actual browser DOM

Updates the DOM when state/props change

React and react-dom work together like a brain (react) and a hand (react-dom)—react decides what to show, and react-dom makes it appear on screen.

Why separate them?

The reason React and ReactDOM were split into two libraries was due to the arrival of **React Native** (A react platform for mobile development).

React components are such a great way to organize UI that it has now spread to mobile to react is used in web and in mobile. react-dom is used only in web apps.

How to Change Port Number

You can change the port number for your React application using a few methods:

1. Edit vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3001
  }
})
```

2. Using Command Line

Run directly:

```
npm run dev -- --port 3001
```

3. Using .env

Create .env

```
VITE_PORT=3001
```

But Vite **does not automatically use PORT=** like CRA.

You need to use it in vite.config.js

Example:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  server: {
    port: Number(process.env.VITE_PORT)
  }
})
```

Understanding Purpose

What are Static Files in public/?

Static files are files that:

- do not change during runtime
- are served directly to browser
- are not processed or optimized by Vite

Examples of Static Files

Static File	Purpose
favicon.svg	Website tab icon
robots.txt	SEO configuration
PDF files	Downloadable documents
Static images	Logos/backgrounds

public/ vs src/assets/

public/	src/assets/
Static files	Component-related files
Direct URL access	Imported into React components
No Vite optimization	Optimized by Vite
Good for favicon, PDFs	Good for images/icons used in components

Example

Store in public/

public/logo.png

Use directly:

```

```

Store in src/assets/

src/assets/logo.png

Import before use:

```
import logo from './assets/logo.png'  
<img src={logo} />
```

Annexure II

Map

In React, the Map method used to traverse and display a list of similar objects of a component.

Often, we find ourselves needing to take an array and modify every element in it. Use **.map()** **whenever you need to update data inside an array (by mapping over it!).**

A map is not the feature of React. Instead, it is the standard JavaScript function that could be called on any array.

The map() method creates a new array by calling a provided function on every element in the calling array.

Syntax

```
array.map(callback[, thisObject]);
```

Parameter Details

- callback – Function that produces an element of the new Array from an element of the current one.
- thisObject – Object to use as this when executing callback.

Return Value

Returns the created array.

Filter

filter() loops through data, and filters out data that doesn't match the criteria that we set. So, It's the process of looping through an array and including or excluding elements inside that array based on a condition that you provide.

It is also built in JavaScript function.

Example-1 :Create react app to display all array elements in h2 tag using **map function**.

Map1.jsx

```
import React from 'react';

function Map1() {
  const arr=[1,2,3,4,5];
  return (
    <div>
      <h1>Example of mapping</h1>
      {
        arr.map((value)=>
          {
            return <h2>Array Element= {value}</h2>
          }
        )
      }
    </div>
  )
}
```

```

    })    }
  </div>  )}
export default Map1

```

Output:

Example of mapping

Array Element= 1

Array Element= 2

Array Element= 3

Array Element= 4

Array Element= 5

Example-2: Write a program to create ReactJS application having an array of strings and convert it in Uppercase using MAP method.

In Arraymap.jsx file:

```

import React from 'react'
const Arraymap = () => {
  const arr=["a","b","c","d","e"];
  return (
    <div>
      <h1>map function</h1>
      { arr.map((value)=>
        {
          return <p>array values= {value.toUpperCase()}</p>
        } ) }
    </div>
  ) }
export default Arraymap

```

Output: (Call Arraymap.jsx in App.jsx)

map function

array values= A

array values= B

array values= C

array values= D

array values= E

Example-3: Create react app which displays images using map function.

Product.jsx

```

import img1 from "./img1.jpg" //import image from same folder
import img2 from "./img2.jpg" //import image from same folder

function Product() {
  const images=[{id:1, pic:img1}, { id:1,pic:img2}];

  return (

```

```

    <div>
      {
        images.map((val) => {
          return <img src={val.pic} height="200px" width="200px" alt="logo" />
        })
      }
    </div>
  ) }
}
export default Product

```

Output: import Product.jsx in App.jsx. This will display two images.

Example 4: Create we have apply filter to skip digit “3” from the array and display all remaining digits of the array.

Filt1.jsx

```

import React from 'react';

function Filt1() {
  const arr=[1,2,3,4,5,3,7,3,9];
  const newarr = arr.filter(num => num!==3)
  var arr1 = arr.join(", "); //join each element of array by “,”
  var arr2 = newarr.join(", "); //join each element of array by “,”
  return (
    <div>
      <h1>Array elements before applying filter <span style={{color:"red"}}> {arr1}
    </span> </h1>
      <h1>Array elements after applying filter <span style={{color:"red"}}> {arr2} </span>
    </h1>
    </div>
  )}
}
export default Filt1

```

App.jsx

```

import Filt1 from "./Filt1";

function App() {
  return (
    <div>
      <Filt1/>
    </div>
  ) }
}
export default App;

```

Output:

Array elements before applying filter 1, 2, 3, 4, 5, 3, 7, 3, 9

Array elements after applying filter 1, 2, 4, 5, 7, 9

Example 5: Create react app to perform the tasks as asked below.

Add array of 5 objects with properties Name and Age. Check if age is greater than 50 then display the person name of who are greater than 50 age.

M1.jsx

Using only map function to fulfil condition and to display values.

```
import React from 'react'
function M1() {
  const people = [{ name: 'ABC', age: 31},{ name: 'XYZ', age: 55},{ name: 'PQR', age: 36},
  { name: 'EFG', age: 69},{ name: 'DEF', age: 34} ];
  return (
    <ul>
      {
        people.filter((person) => (person.age > 50)).map((person)=>{
          return( <h3>{person.name}</h3> )})
        }
      </ul>
    ) }
  export default M1
```

Example 6: Create react app to display name of students in h3 tag of CSE branch from given object.

M2.jsx

```
import React from 'react'
function M2() {
  let students = [
    { id: "001", name: "A1", Branch: "CSE" }, { id: "002", name: "A2", Branch: "CE" },
    { id: "003", name: "A3", Branch: "CSE" }, { id: "004", name: "A4", Branch: "CSE" },
    { id: "005", name: "A5", Branch: "IT" } ]
  return (
    <div>
      {
        students.filter((student) => (student.Branch === "CSE")).map((student)=>{
          return <h3>{student.name}</h3>
        }
      })
    </div>
  ) }
  export default M2
```

Example 7

Write React code to filter out the number greater than 3 using Map method and Filter method to filter the array.

[Discuss the difference between two method's output and compare console.]

```
function Task() {
  const arr=[1,2,3,4,5,3,6,4,3,1];
  var t = arr.map((val,i) => {
    if (val > 3) return (
      <h2 key={i}>{val}</h2>
    )
  })
  console.log(t)

  var t1 = arr.filter((val,i) => {
    if (val > 3) return (
      <h2 key={i}>{val}</h2>
    )  })
  console.log(t1)

  var t3=t1.map((val)=>{return(
    <h2 key={i}>{val}</h2>)
  })
  console.log(t3)

  return(
    <>
    <div> array values ={t}</div>
    <p>{t1.join(', ')}</p>
    <h3>{t3}</h3>
    </>
  )}
  export default Task
```